

A grid, and the same job written twice

Six tasks, sixteen minutes, finished in the room. Tasks 1 to 4 are today's material applied directly. Tasks 5 and 6 ask you to write one job as a loop and then as a comprehension, so that you can see with your own eyes that they are the same thing.

Guided lab

16 minutes

Not graded

Not a prerequisite for session 6

[Before you start](#)[1 · slices](#)[2 · the copy](#)[3 · read a grid](#)[4 · row totals](#)[5 · twice](#)[6 · find it](#)[If you finish early](#)[Reflection](#)

Before you start

Open the provided `lab02.ipynb` and run the first cell before anything else — it holds this grid, and tasks 3 onward all use it:

cell 1 – run this first

PYTHON

```
grid = [[4, 7, 2, 9],  
        [1, 8, 6, 3],  
        [5, 0, 7, 2]]
```

Three rows, four columns. Write those two numbers at the top of your page — half of the mistakes in this lab come from losing track of which is which.

TASK 1 · 2 MINUTES

Predict, then run

1

No writing. Just say what each line gives before you execute it.

Write your answer for each line on paper *first*. Then run the cell and compare. The point of this task is to find out where your mental model is wrong while it is cheap to find out.

predict all eight, then run

PYTHON

```
a = [10, 20, 30, 40, 50, 60]
```

```
a[0]
```

```
a[-1]
```

```
a[1:4]
```

```
a[:2]
```

```
a[4:]
```

```
a[-2:]
```

```
len(a[1:4])
```

```
a[2:2]
```

10 · 60 · [20, 30, 40] · [10, 20] · [50, 60] · [50, 60] · 3 · []

The last one surprises people: from gap 2 to gap 2 is no distance, so the slice is empty. And note that

`len(a[1:4])` is $4 - 1 = 3$ without any counting — that is the whole point of the fencepost model.

TASK 2 · 2 MINUTES

Break it, then fix it

2

Run this exactly as written and confirm that it prints what the comment says. Then change one line so that `original` is left untouched, and run it again.

```
original = [1, 2, 3]
copy = original
copy[0] = 99

print(original)    # [99, 2, 3] ← and you never touched original
print(copy)        # [99, 2, 3]
```

Fix it so the output is `[1, 2, 3]` and `[99, 2, 3]`. One character changes.

TASK 3 · 2 MINUTES

Read the grid

3

Using the grid from cell 1, write one line each that produces:

- a the whole middle row → `[1, 8, 6, 3]`
- b the number 6 → it is in row 1, column 2
- c the number of rows, and the number of columns, as two separate numbers
- d the last number in the last row, without writing 2 or 3 as an index



For (d)

Negative indexing works at both levels. `grid[-1]` is the last row, and it is itself a list.

TASK 4 · 4 MINUTES

row_totals

The task from part 5 of the lecture. If you did it there, do it again without looking.

4

Receives: a grid. **Returns:** a list with one total per row.

test it with this

PYTHON

```
print(row_totals(grid))          # [22, 18, 14]
print(row_totals([[1, 1], [2, 2]])) # [2, 4]
print(row_totals([]))           # []
```

Two accumulators, one inside the other. The list of results is created before the outer loop; the running total is created fresh at the top of each row.

PYTHON

```
def row_totals(g):
    results = []
    for row in g:
        running = 0
        for value in row:
            running = running + value
        results.append(running)
    return results
```

If your answer came out as one number instead of a list, `running = 0` is probably above the outer loop instead of inside it.

TASK 5 · 3 MINUTES

The same job, twice

Once as a loop you already know, once as a comprehension. Both in the same cell.

5

Take the list `temps = [21, 25, 18, 30, 22]`, in Celsius, and produce the same list in Fahrenheit.

fill in both

PYTHON

```
temps = [21, 25, 18, 30, 22]

# version A – the loop
fahrenheit_a = []
# ... your three lines here

# version B – the comprehension, one line
fahrenheit_b = ...

print(fahrenheit_a)
print(fahrenheit_b)
print(fahrenheit_a == fahrenheit_b)  # must be True
```



The last line is the task

Getting `True` is the whole point. A comprehension is not a new idea and not a clever trick — it is the loop above it, written on one line. When you meet NumPy on Friday you will drop both, but you will keep this way of thinking: *do this to every element*.

TASK 6 • 3 MINUTES

Find where it is

The one that needs positions rather than values — and the one worth the most on Friday.

6

Receives: a grid. **Returns:** the largest number in it, together with its row and column.

test it with this

PYTHON

```
print(find_largest(grid))  # 9 at row 0 column 3
```

Return the three values however you like — a tuple `(value, row, col)` is the tidiest, and it is the form the assert in the notebook checks for.



Two traps, both deliberate

Starting your "largest so far" at zero breaks the moment a grid is all negative — try `[[-4, -7], [-1, -9]]` and check you get `-1`. And you need `range` here rather than `for row in grid`, because a value alone cannot tell you where it was.

If you finish early

- A `column_totals(g)` — one total per column instead of per row, giving `[10, 15, 15, 14]` for our grid. Harder than it looks: you must loop over columns on the outside and rows on the inside.
- B `transpose(g)` — turn rows into columns, so our 3×4 grid becomes 4×3 . On Friday this is `a.T`, one character. Doing it by hand once is how you will understand what that character costs.
- C **Brightness.** Treat the grid as a tiny grayscale image where 0 is black and 9 is white. Write `brighten(g, amount)` returning a new grid with `amount` added to every value, capped at 9. You have just written your first image operation.
- D **Read a traceback.** The last cell of the notebook runs `print(values[3])` on a three-item list, on purpose. Before fixing it, find the *last line* of the red message and the line number it points to, and write in one sentence which exact clue told you what went wrong. Reading the bottom of a traceback first will save you more time this semester than any shortcut key.

Reflection



Write it in the notebook, in your own words

Why are lists, indexing and loops worth mastering *before* NumPy arrives on Friday? Give one concrete example of a later AI operation — a layer, a loss, an image — that would be hard to follow if `grid[r][c]` and an accumulator were still mysterious. Two or three sentences.